

Laboratorio EDD

MANUAL TECNICO

Alvaro Morales



Manual tecnico

Main.f90:

El archivo main.f90 contiene el programa principal que interactúa con los módulos jsonReaderModule y module_sha256 para realizar diversas operaciones, como iniciar sesión, cargar datos, generar informes y salir del programa. Aquí tienes una descripción de las partes principales del código:

```
program main
  use jsonReaderModule  Cannot open module file 'jsonreader
  use module_sha256
  implicit none
  type(jsonReader) :: reader
  integer :: option, idSucursalInicio, idSucursalFin
  integer*8 :: dpi

  character(len=100) :: usuarioDefault, usuario, filename
  character(len=100) :: passwordDefault, password
  usuarioDefault = "admin"
  passwordDefault = "admin"
  passwordDefault = sha256(passwordDefault)
```

JsonReader.f90:

aEl módulo jsonReaderModule proporciona las siguientes funciones y subrutinas principales:

- readJsonTecnicos: Lee datos de técnicos desde un archivo JSON y los almacena en una tabla hash.

Merkle.f90:

El módulo merkle_tree define las siguientes estructuras de datos:

- data: Representa los datos almacenados en el árbol Merkle, incluyendo información como identificador de origen y destino, direcciones y costo entre las sucursales.
- hash_node: Representa un nodo del árbol Merkle, que contiene un valor hash, referencias a sus nodos hijos y una referencia a los datos asociados.
- merkle: Representa el árbol Merkle en sí, con un puntero a la raíz del árbol, referencias al primer y último nodo de datos, y un contador de posición.

```
module merkle_tree
  use module sha256      Cannot open module file 'module_sha256.mod' for rea
  implicit none
  integer :: uid = 1

  type data
    integer :: uid
    character(:, allocatable) :: id_origin, address_origin
    character(:, allocatable) :: id_destination, address_destination
    character(:, allocatable) :: cost_between, hash_value
    type(data), pointer :: next => null()
  end type data

  type hash_node
    integer :: uid
    character(:, allocatable) :: hash
    type(hash_node), pointer :: left => null()
    type(hash_node), pointer :: right => null()
    type(data), pointer :: dataref => null()
  end type hash_node

  type merkle
    type(hash_node), pointer :: top_hash => null()
    type(data), pointer :: data_head => null()
    type(data), pointer :: data_tail => null()
  end type merkle
end module merkle_tree
```

Routes.f90:

- edge: Representa una arista en el grafo. Contiene información como identificador, peso, identificador del nodo padre y cantidad de impresoras.
- edge_list: Representa una lista de aristas, con métodos para agregar aristas ordenadas, eliminar aristas, comprobar si la lista está vacía, fusionar listas y actualizar los pesos de las aristas.

```
type edge
  integer :: id
  integer :: weight
  integer :: parent_id
  integer :: printers
  type(edge), pointer :: next => null()
  type(edge), pointer :: prev => null()
end type edge

type edge_list
  type(edge), pointer :: head => null()
  type(edge), pointer :: tail => null()
  contains
    procedure :: add_sorted
    procedure :: pop
    procedure :: is_empty
    procedure :: merge
    procedure :: add_weight
    procedure :: get_size
  end type edge_list
```

Sha256.f90:

- 1.Función sha256: Toma una cadena de caracteres como entrada y devuelve su hash SHA-256. Esta función utiliza la función interna sha256b con el valor de swap igual a 1.
- 2.Función dirty_sha256: Similar a sha256, pero utiliza sha256b con el valor de swap igual a 0.

```
function sha256(str)
  character(len=64) :: sha256
  character(len=*), intent(in) :: str
  sha256 = sha256b(str, 1)
end function sha256
```

```
function sha256b(str, swap)
  character(len=64) :: sha256b
  character(len=*), intent(in) :: str
  integer, intent(in) :: swap
  integer(kind=c_int64_t) :: length
  integer(kind=c_int32_t) :: temp1, temp2, i
  integer :: break, pos0
  integer(kind=c_int32_t) :: h0_ref(8), k0_ref(64)
  integer(kind=c_int32_t) :: h0(8), k0(64), a0(8), w0(64)
  data (h0_ref(i), i=1,8)/&
    & z'6a09e667', z'bb67ae85', z'3c6ef372', z'a54ff53a', z'510e527f', z'9b05688c', z'1f83d9ab',
  data (k0_ref(i), i=1,64)/&
    & z'428a2f98', z'71374491', z'b5c0fbcf', z'e9b5dba5', z'3956c25b', z'59f111f1', z'923f82a4',
    & z'd807aa98', z'12835b01', z'243185be', z'550c7dc3', z'72be5d74', z'80deb1fe', z'9bdc06a7',
    & z'e49b69c1', z'efbe4786', z'0fc19dc6', z'240ca1cc', z'2de92c6f', z'4a7484aa', z'5cb0a9dc',
    & z'983e5152', z'a831c66d', z'b00327c8', z'bf597fc7', z'c6e00bf3', z'd5a79147', z'06ca6351',
    & z'27b70a85', z'2e1b2138', z'4d2c6dfe', z'53380d13', z'650a7354', z'766a0abb', z'81c2c92e',
    & z'a2bfe8a1', z'a81a664b', z'c24b8b70', z'c76c51a3', z'd192e819', z'd6990624', z'f40e3585',
    & z'19a4c116', z'1e376c08', z'2748774c', z'34b0bcb5', z'391c0cb3', z'4ed8aa4a', z'5b9cca4f',
    & z'748f82ee', z'78a5636f', z'84c87814', z'8cc70208', z'90befffa', z'a4506ceb', z'bef9a3f7',
  h0 = h0_ref
```

Sucursales.f90:

- Definición de tipos: Define dos tipos de datos: sucursal y node. La estructura sucursal contiene información sobre una sucursal, incluyendo su identificador, número de impresoras en mantenimiento, departamento, dirección y una tabla hash de técnicos registrados en esa sucursal. La estructura node representa un nodo en el árbol AVL y contiene un valor de tipo sucursal, un identificador único (uid), una altura y enlaces a los nodos hijo izquierdo y derecho.

```
module module_AvlSucursales

  type sucursal
    integer :: id
    integer :: impresorasMantenimiento
    character(len=50) :: departamento
    character(len=50) :: direccion
    character(len=50) :: password
    type (hash) :: hash !Técnicos registrados en
  end type sucursal

  type :: node
    type (sucursal) :: value
    integer :: uid
    integer :: height
    type(node), pointer :: left => null()
    type(node), pointer :: right => null()
  end type
```

TablaHash.f90:

1. Define dos tipos de datos: HashTable, que contiene información sobre un técnico, como el DPI, nombre, apellido, dirección, género, número de teléfono y cantidad de trabajos realizados; y hash, que representa la tabla hash para almacenar objetos de tipo HashTable. La tabla hash se implementa como un arreglo dinámico de tipo HashTable.

```
type hashTable
  integer(kind=8) :: dpi
  character(:), allocatable :: name, last_name, address, gender, phone
  integer :: trabajos_realizados
end type hashTable

type :: hash
  integer :: n ! Number of elements
  integer :: m ! Table size
  integer :: mini, maxi ! Min and max precentages (el mini que vamos a usar es de 50%)
  type(hashTable), dimension(:), allocatable :: hashT ! Hash table
contains
  procedure :: init
  procedure :: dispersion
  procedure :: collision
  procedure :: insert
  procedure :: rehashing
  procedure :: search
  procedure :: showDataHash
  procedure :: showList
  procedure :: grafico
end type hash
```

Blockchain.f90:

- Definición de tipos: Define dos tipos de datos: sucursal y node. La estructura sucursal contiene información sobre una sucursal, incluyendo su identificador, número de impresoras en mantenimiento, departamento, dirección y una tabla hash de técnicos registrados en esa sucursal. La estructura node representa un nodo en el árbol AVL y contiene un valor de tipo sucursal, un identificador único (uid), una altura y enlaces a los nodos hijo izquierdo y derecho.

```
module module_blockchain
  implicit none

  type block
    integer :: index
    character(len=25) :: timestamp
    character(len=50) :: data
    integer :: nonce
    character(len=32) :: previous_hash
    character(len=256) :: rootmerklel
    character(len=32) :: hash
    type(block), pointer :: next => null()
  end type block

  type blockchain
    type(block), pointer :: head
    type(block), pointer :: tail
    integer :: index = 0
  end type blockchain
```